Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

Oct 30, 2020 · 6 min read · ✨ · Listen

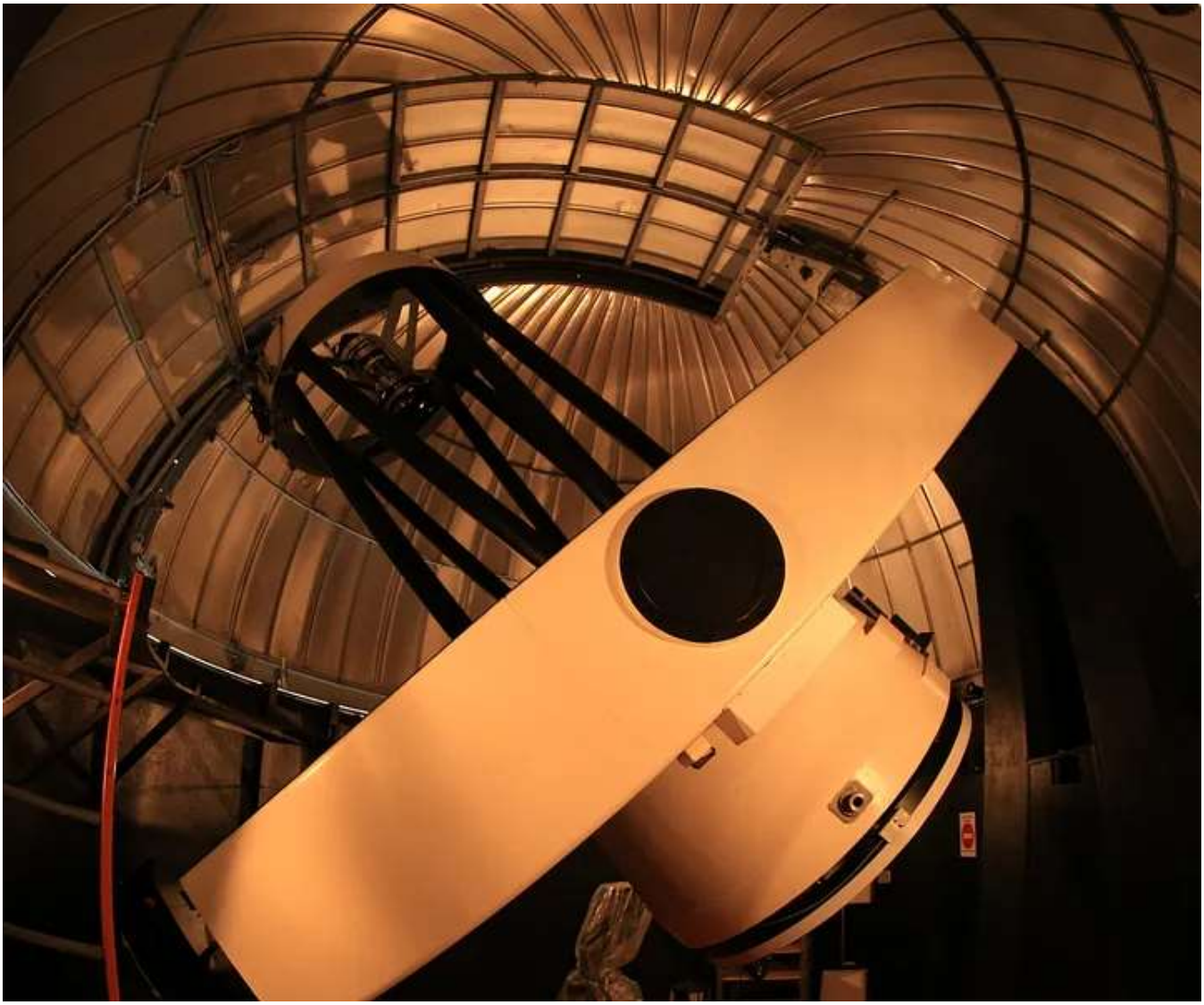
Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — Asteroid Project (Part 4)

Part 24 of the tutorial series provides an overview of the current science project's state. We create an Open Source Python library for our project that shall be sustainable and extensible for future science work.

28



One of the two telescopes at the C2PU (https://www.oca.eu/fr/c2pu-accueil). C2PU is a telescope site near Nice, France and is associated with the Université Côte d'Azur. Each telescope has a diameter of 1 m and has several CCD cameras, filters, etc. A 1 m telescope can be absolutely sufficient for NEO surveys and observations. Image by Author (T. Albin).

Preface

This is the 24th part of my Python tutorial series "Space Science with Python". All codes that are shown in the tutorial sessions are uploaded on GitHub. The shown Python library SolarY can be found on GitHub, too. Enjoy!

Project Recall and Purpose

Last time we discussed how one can develop sustainable and (relatively) accurate and well tested code: e.g., by using a Test Driven Development (TDD) approach.

Asteroid Project (Part 2) — Test Driven Development

Part 22 of the tutorial series continues with the second part of our science project. Before we dive deeper into Python...

towardsdatascience.com

Space Science with Python — Asteroid Project (Part 3)

Part 23 of the tutorial series continues with the third part of our science project. Today we will go in detail...

towardsdatascience.com

Let's recall our project idea and purpose briefly:

- Near-Earth Objects (NEOs) approach the Sun at least in a distance of 1.3 Astronomical Units (AU). This border has been set artificially.
- Consequently, a certain sub-population of NEOs approach our Earth quite closely; may cross our home planet's orbit; or even collide with it like the Tunguska Event in 1908 or the Meteor of Chelyabinsk in the year 2013.
- Continuous sky surveys have already identified and classified over 24,000 NEOs!
- Recent NEO population models suggest that there are hundred thousands of objects more to discover.
- The scientific question and project idea here that rises: *Can we optimize the observation strategy of our telescopes? Where shall we point our telescopes at? And what are the ideal telescope and camera settings for a successful observation campaign?*

A simulation work flow

How can we answer this scientific / technical questions?

Synthetic NEOs

Well, first of all we will need a state of the art NEO population model that provides an unbiased distribution of sizes and orbital elements. Granvik et al. (2018) provide a great dataset that is being described in their paper (free available on arXiv):

Debiased orbit and absolute-magnitude distributions for near-Earth objects

We provide a four-dimensional debiased orbit and absolute-magnitude distribution for near-Earth objects. * The model...

www.sciencedirect.com

On his website, Granvik provides free access to over 800,000 synthetically generated NEOs ...

Index of /home/mgranvik/data/Granvik+_2018_Icarus

Edit description

www.mv.helsinki.fi

... that shall be the data fundament of our simulation task.

Modelling the observation / detection process

Based on the NEO data we will need to simulate the entire object detection pipeline. Picking one NEO, the pipeline must contain:

- Computing the position of the NEO and Earth
- Based on the NEO's size, albedo and distance to Earth: compute its apparent magnitude
- (Optional: considering the telescope's position on Earth and the atmospheric attenuation)
- Converting the apparent magnitude to its corresponding irradiance (Watt per square-meter)
- Modelling the telescope's optics and considering its CCD settings like: focal length, aperture, pixel size, quantum efficiency of the chip, exposure time, etc.
- Computing the signal and also noise that is based on: thermals, statistical scattering, hot / dead pixel, over-exposure etc.

- Finally determine the Signal-to-Noise ratio of a NEO and check whether it is detectable or not. Re-do the simulation for different time stamps.

This modelling shall lead to the conclusion whether a NEO has been detected at a particular (simulated) observation night, or not. The detectability of a NEO varies based on the apparent magnitude, its movement on the CCD chip and many other effects. At the end, we may get a binary result for a multi-dimensional problem: *Was the NEO detected or not?*

After modelling the detection pipeline one can then simulate miscellaneous telescope settings, CCD operation modes and so on, to determine how many objects can be detected within a particular time frame. After all, this project leads to a simulation tool, but a definitive answer to a complex problem.

Think in products, not in use cases or prototypes ...

Either in private coding projects, in science and academia or in industry … some developers and free time coders like to see fast results. They develop and deploy, what they call, prototypes for a "quick win". The resulting dashboard, simulation or data scientific insights may impress at first glance … but resulting technical debts and missing documentations lead to a "dead end project" with low to no maintainability and extensibility possibilities.

Let's avoid this **Use Case / Prototype** thinking and let's create a long-term **Product** that may help us, open source enthusiasts and space scientists to re-use generic and commonly used functions in the future.

The result: Our Asteroid Project shall lead to a new Python library that will be a backbone for this and future space science related projects called **SolarY**.

In the last couple of days I already started setting up a repository, some structural elements and modules. The state as it is presented here may only be valid for a few days before new git-commits will extend the library. A first version shall be released this year that shall help us to fulfil our scientific goal.

ThomasAlbin/SolarY

You can't perform that action at this time. You signed in with another tab or window. You signed out in another tab or...

github.com

To avoid third party dependencies this library shall use the Python 3 standard libraries as often as possible. Until now, only *pytest* is currently being used for the TDD approach.

The current structure of SolarY:

```
SolarY
+-- solary
|   +-- _config
|   |   +-- __init__.py
|   |   +-- constants.ini
|   +-- asteroid
|   |   +-- __init__.py
|   |   +-- physp.py
|   +-- general
|   |   +-- __init__.py
|   |   +-- astrodyn.py
|   |   +-- photometry.py
|   |   +-- vec.py
|   +-- neo
|   |   +-- _data
|   |   |   +-- __init__.py
|   |   +-- _databases
|   |   |   +-- __init__.py
|   |   +-- __init__.py
|   |   +-- data.py
|   +-- tests
|   |   +-- _temp
|   |   |   +-- __init__.py
|   |   +-- test_SUBMODULE_PY.py
|   |   +-- __init__.py
|   +-- __init__.py
+-- MISC
```

- ***SolarY*** Main repository folder on GitHub. It contains the future Python module ***solary*** and miscellaneous (***MISC***) auxiliary files like the license, the MANIFEST.in or the setup.py
- ***solary*** Main Python library folder

- ***_config*** This folder contains configuration files that store information like constants (e.g., gravitational constant, magnitude reference, etc.)
- ***asteroid*** This folder contains Python scripts that are related to asteroids. Until now, one Python script has been created that contains equations to derive intrinsic physical parameters of these objects
- ***general*** Vector related functions, photometric functions (e.g., to compute the apparent magnitude of objects, or to convert the magnitude to irradiance) as well as astro-dynamical formulae (Tisserand parameter) are located here
- ***neo*** Until now, this folder contains functions to download the NEODyS-2 data and shall also contain a function to download and verify model data. The data can be stored in any local folder or within the library itself (***_data***). Based on the data, an SQLite database can be created for further data analytical purposes (***_databases***)
- ***tests*** This folder contains all TDD based scripts. The structure of each Python script: ***test_SUBMODULE_PY.py*** (example: test_general_vec.py). Data download tests are temporarily stored in ***_temp***

Future sub-modules could be added for ***comet***s, ***meteoroid***s, ***meteor***s, cosmic ***dust*** or also in-situ ***spacecraft*** missions and data.

Next steps

In the next couple of days and weeks SolarY will be extended. Classes will be added for NEOs, Asteroids, and Telescopes that contain all relevant settings and information. The simulation pipeline will be based on the SolarY sub-modules.

The NEO model data contain orbital elements. However, to compute the apparent magnitude of these objects the state vectors w.r.t. the Sun and our home planet are needed. As described in the first tutorial sessions of *Space Science with Python* the NASA toolkit SPICE and the corresponding Python wrapper *spiceypy* are the best solutions to compute these data. Adding a SPICE dependency shall be avoided, thus, a small side project is added to outsource the SPICE routines outside of SolarY. More details of this “SPICE outsourcing” will follow in the next tutorial.

Thomas

Twitter: MrAstroThomas

Space Science Python Open Source Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play